SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I P.Jyothika(192311156)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Priyanga.B

Rubrics for Evaluation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	***Marks (100)***
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline A system must process a live video stream to:

A. Capture frames from a camera

B. Detect faces in each frame

C. Recognize faces using a pre-trained dataset D. Display bounding boxes and labels in real time Constraints:

- **Must operate continuously**
- **Should minimize processing delay**
- **Should handle multiple faces**

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure. ***Constraints***

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Pseudocode

```
BEGIN FaceDetectionRecognitionPipeline   FUNCTION LoadResources()
faceCascade = LOAD_HAAR_CASCADE("haarcascade_frontalface.xml")
recognizer  = LOAD_TRAINED_RECOGNIZER("face_model.yml")
labelMap    = LOAD_LABELS("labels.txt")
    RETURN faceCascade, recognizer, labelMap
 END FUNCTION
 FUNCTION PreprocessFrame(frame)       grayFrame =
CONVERT_TO_GRAYSCALE(frame)       grayFrame =
EQUALIZE_HISTOGRAM(grayFrame)
    RETURN grayFrame
 END FUNCTION
 FUNCTION DetectFaces(grayFrame, faceCascade)       faceList
= faceCascade.DETECT_MULTISCALE(grayFrame,
                  scaleFactor=1.1, minNeighbors=5)
    RETURN faceList     // returns ALL faces -> handles multiple faces
 END FUNCTION
 FUNCTION RecognizeFace(grayFrame, faceRect, recognizer, labelMap)
faceROI = CROP(grayFrame, faceRect)       faceROI = RESIZE(faceROI,
200, 200)
```

```
    label, confidence = recognizer.PREDICT(faceROI)
IF confidence < THRESHOLD THEN          name =
labelMap[label]      ELSE
        name = "Unknown"
    END IF
    RETURN name
  END FUNCTION
MAIN:
    faceCascade, recognizer, labelMap = LoadResources()
videoCapture = OPEN_CAMERA(0)
    WHILE videoCapture.IS_OPENED() DO       // continuous operation
frame = videoCapture.READ_FRAME()
        IF frame IS NULL THEN
           BREAK          END IF
grayFrame = PreprocessFrame(frame)
        faceList  = DetectFaces(grayFrame, faceCascade)
        FOR EACH face IN faceList DO        // handles multiple faces per frame
name = RecognizeFace(grayFrame, face, recognizer, labelMap)
           DRAW_RECTANGLE(frame, face)
           PUT_TEXT(frame, name, face.position)
        END FOR
        DISPLAY(frame)
        IF KEY_PRESSED("q") THEN
           BREAK
        END IF
    END WHILE
    videoCapture.RELEASE()
    CLOSE_ALL_WINDOWS()
END FaceDetectionRecognitionPipeline
```

Justification

The single-pass DETECT_MULTISCALE call combined with the FOR EACH loop over faceList satisfies the multiple-faces constraint without re-running detection per face. Working on a grayscale, equalized frame keeps detection fast, minimizing delay. The outer WHILE loop with a null-frame/key-press exit condition ensures continuous operation until explicitly stopped.

Modular functions (LoadResources, PreprocessFrame, DetectFaces, RecognizeFace) separate concerns, making the pipeline easy to extend or optimize independently.

2. OCR Pipeline for Document Processing A
document processing system must:

A. Read input images containing printed text
B. Enhance image quality (noise removal, contrast improvement)
C. Segment text regions
D. Extract and display recognized text Constraints:

- **Input images may be noisy and low contrast**
- **Batch processing required**

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Constraints

- Input images may be noisy and low contrast
- Batch processing required

Pseudocode

```
BEGIN OCRPipeline
  FUNCTION EnhanceImage(image)       grayImage   =
CONVERT_TO_GRAYSCALE(image)
     denoised    = DENOISE(grayImage)             // handles noisy input
contrastImg = APPLY_CLAHE(denoised)           // handles low contrast
binaryImg   = ADAPTIVE_THRESHOLD(contrastImg)
     RETURN binaryImg
  END FUNCTION
  FUNCTION SegmentTextRegions(binaryImg)
dilatedImg  = DILATE(binaryImg, kernel)       contours
= FIND_CONTOURS(dilatedImg)       textRegions =
EMPTY_LIST()
     FOR EACH contour IN contours DO
IF AREA(contour) > MIN_AREA THEN
            textRegions.APPEND(BOUNDING_BOX(contour))
          END IF
     END FOR
     RETURN textRegions
  END FUNCTION
  FUNCTION ExtractText(image, textRegions)
resultText = ""
     FOR EACH region IN textRegions DO
roi  = CROP(image, region)
```

```
        text = OCR_ENGINE.RECOGNIZE(roi)
resultText = resultText + text + NEWLINE
    END FOR
    RETURN resultText
  END FUNCTION   MAIN:      imageList =
LOAD_IMAGES_FROM_FOLDER("input_folder")   // batch processing
    FOR EACH image IN imageList DO          enhancedImg    =
EnhanceImage(image)         textRegions    =
SegmentTextRegions(enhancedImg)          recognizedText  =
ExtractText(enhancedImg, textRegions)
SAVE_TEXT(recognizedText, image.name + "_output.txt")
        DISPLAY(recognizedText)
    END FOR
END OCRPipeline
```

Justification

DENOISE and APPLY_CLAHE directly counter the stated noisy/low-contrast input constraint before any segmentation is attempted, so downstream stages work on cleaner data. Adaptive thresholding handles uneven illumination better than a single global threshold. The outer FOR EACH image IN imageList loop processes the whole folder sequentially, satisfying the batchprocessing requirement, while each stage (enhance, segment, extract) remains a separate reusable function.

3. Motion Detection and Tracking System A surveillance system must:

A. Read video input
B. Detect motion between consecutive frames
C. Track moving objects
D. Highlight detected motion regions Constraints:

- **Should handle dynamic background**
- **Must operate in near real-time**

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

Constraints

- Should handle dynamic background
- Must operate in near real-time

Pseudocode

```
BEGIN MotionDetectionTracking
  FUNCTION InitializeBackgroundModel()       bgSubtractor =
CREATE_BACKGROUND_SUBTRACTOR_MOG2()   // adapts to dynamic
background
     RETURN bgSubtractor
  END FUNCTION
  FUNCTION DetectMotion(frame, bgSubtractor)       fgMask =
bgSubtractor.APPLY(frame)       fgMask =
MORPHOLOGICAL_OPEN(fgMask)     // remove noise       fgMask
= MORPHOLOGICAL_DILATE(fgMask)   // fill gaps       contours =
FIND_CONTOURS(fgMask)       motionRegions = EMPTY_LIST()
     FOR EACH contour IN contours DO
        IF AREA(contour) > MIN_MOTION_AREA THEN
motionRegions.APPEND(BOUNDING_BOX(contour))
        END IF
     END FOR
     RETURN motionRegions
  END FUNCTION
  FUNCTION TrackObjects(motionRegions, trackedObjects)
FOR EACH region IN motionRegions DO
        matchedID = MATCH_TO_EXISTING(region, trackedObjects)
        IF matchedID EXISTS THEN
           UPDATE_POSITION(trackedObjects[matchedID], region)
ELSE
           newID = GENERATE_NEW_ID()
trackedObjects[newID] = region
        END IF
     END FOR
     RETURN trackedObjects
  END  FUNCTION      MAIN:            videoCapture    =
OPEN_VIDEO("input_video.mp4")       bgSubtractor  =
InitializeBackgroundModel()              trackedObjects  =
EMPTY_MAP()                                    WHILE
videoCapture.IS_OPENED()  DO                   frame  =
videoCapture.READ_FRAME()
        IF frame IS NULL THEN
```

```
        BREAK        END IF        motionRegions  =
DetectMotion(frame, bgSubtractor)        trackedObjects =
TrackObjects(motionRegions, trackedObjects)
      FOR EACH obj IN trackedObjects DO
        DRAW_RECTANGLE(frame, obj.position)
        PUT_TEXT(frame, "ID:" + obj.id, obj.position)
      END FOR
      DISPLAY(frame)
      IF KEY_PRESSED("q") THEN
        BREAK
      END IF
    END WHILE
    videoCapture.RELEASE()
CLOSE_ALL_WINDOWS()
END MotionDetectionTracking
```

Justification

MOG2 maintains an adaptive statistical background model, so gradual scene changes (lighting drift, swaying trees) are absorbed rather than flagged as motion — directly satisfying the dynamic-background constraint. Morphological opening/dilation is computationally cheap and removes pixel noise before contour extraction, keeping per-frame cost low for near real-time operation. Simple nearest-match tracking (MATCH_TO_EXISTING) avoids expensive reidentification, maintaining object identity across frames efficiently.

4. Vehicle Counting System using Video A
traffic monitoring system must:

A. Read video frames
B. Detect vehicles entering a region of interest (ROI)
C. Track movement across frames D. Count vehicles and display count Constraints:

- **Avoid double counting**
- **Handle multiple vehicles simultaneously**

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions. ***Constraints***

- Avoid double counting
- Handle multiple vehicles simultaneously

Pseudocode

```
BEGIN VehicleCountingSystem
  FUNCTION DetectVehicles(frame, bgSubtractor)
fgMask = bgSubtractor.APPLY(frame)       fgMask
= MORPHOLOGICAL_OPEN(fgMask)
contours = FIND_CONTOURS(fgMask)
vehicleList = EMPTY_LIST()
     FOR EACH contour IN contours DO
        IF AREA(contour) > MIN_VEHICLE_AREA THEN
           vehicleList.APPEND( (CENTROID(contour), BOUNDING_BOX(contour)) )
        END IF
     END FOR
     RETURN vehicleList          // list supports multiple simultaneous vehicles
  END FUNCTION
  FUNCTION TrackVehicles(vehicleList, trackedVehicles)
FOR EACH vehicle IN vehicleList DO
        matchedID = MATCH_NEAREST(vehicle.centroid, trackedVehicles)
        IF matchedID EXISTS THEN
           UPDATE(trackedVehicles[matchedID], vehicle)
ELSE
           newID = GENERATE_NEW_ID()
trackedVehicles[newID] = vehicle
trackedVehicles[newID].counted = FALSE
        END IF
     END FOR
     RETURN trackedVehicles
  END FUNCTION
  FUNCTION CountVehicles(trackedVehicles, countLineY, vehicleCount)
     FOR EACH vehicle IN trackedVehicles DO
        IF vehicle.counted == FALSE AND CROSSES(vehicle.centroid.y, countLineY)
THEN
           vehicleCount = vehicleCount + 1
           vehicle.counted = TRUE      // marks vehicle so it is never counted again
END IF
     END FOR
     RETURN vehicleCount
  END FUNCTION
MAIN:
     videoCapture   = OPEN_VIDEO("traffic_video.mp4")
```

```
    bgSubtractor   = CREATE_BACKGROUND_SUBTRACTOR_MOG2()
trackedVehicles = EMPTY_MAP()      vehicleCount   = 0
    countLineY     = DEFINE_ROI_LINE()
WHILE videoCapture.IS_OPENED() DO
frame = videoCapture.READ_FRAME()
        IF frame IS NULL THEN
           BREAK          END IF          vehicleList     =
DetectVehicles(frame, bgSubtractor)          trackedVehicles =
TrackVehicles(vehicleList, trackedVehicles)
        vehicleCount    = CountVehicles(trackedVehicles, countLineY, vehicleCount)
        DRAW_LINE(frame, countLineY)
        FOR EACH vehicle IN trackedVehicles DO
           DRAW_RECTANGLE(frame, vehicle.boundingBox)
        END FOR
        PUT_TEXT(frame, "Count: " + vehicleCount, position=(10,30))
        DISPLAY(frame)
        IF KEY_PRESSED("q") THEN
           BREAK
        END IF
    END WHILE
    videoCapture.RELEASE()
CLOSE_ALL_WINDOWS()
END VehicleCountingSystem
```

Justification

Each tracked vehicle carries a persistent 'counted' flag that is set to TRUE the instant it crosses the counting line, so subsequent frames skip it — this directly prevents double counting. Using a map/list of tracked vehicles (rather than a single variable) and looping FOR EACH vehicle lets the system detect, track, and count several vehicles in the same frame simultaneously. Background subtraction plus a minimum-area filter keeps detection fast enough for continuous video processing.

5. Facial Expression Recognition System A mobile-based system must:
A. Capture images from a camera
B. Detect faces

C. Extract facial features

D. Classify expressions (happy, sad, neutral)

E. Display results Constraints:

- **Limited computational resources**
- **Real-time response required**

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification. ***Constraints***

- Limited computational resources
- Real-time response required

Pseudocode

```
BEGIN FacialExpressionRecognition
FUNCTION CaptureImage(camera)
frame = camera.READ_FRAME()
    frame = RESIZE(frame, 320, 240)    // reduces load for limited resources
    RETURN frame
  END FUNCTION
  FUNCTION DetectFace(frame, faceCascade)        grayFrame =
CONVERT_TO_GRAYSCALE(frame)                  faceList     =
faceCascade.DETECT_MULTISCALE(grayFrame)
    RETURN faceList, grayFrame
  END FUNCTION
  FUNCTION ExtractFeatures(grayFrame, faceRect)
faceROI  = CROP(grayFrame, faceRect)      faceROI
= RESIZE(faceROI, 100, 100)
    features = COMPUTE_LBP_HISTOGRAM(faceROI)   // fast, lightweight descriptor
RETURN features
  END FUNCTION
  FUNCTION ClassifyExpression(features, classifier)
    expression = classifier.PREDICT(features)   // happy / sad / neutral
    RETURN expression
  END FUNCTION   MAIN:      faceCascade =
LOAD_HAAR_CASCADE("haarcascade_frontalface.xml")      classifier  =
LOAD_TRAINED_MODEL("expression_model.xml")      camera     =
OPEN_CAMERA(0)      WHILE camera.IS_OPENED() DO         frame =
CaptureImage(camera)         faceList, grayFrame = DetectFace(frame,
faceCascade)
```

```
        FOR EACH face IN faceList DO              features   =
ExtractFeatures(grayFrame, face)             expression =
ClassifyExpression(features, classifier)
            DRAW_RECTANGLE(frame, face)
            PUT_TEXT(frame, expression, face.position)
        END FOR
        DISPLAY(frame)
        IF KEY_PRESSED("q") THEN
            BREAK
        END IF
    END WHILE
camera.RELEASE()
    CLOSE_ALL_WINDOWS()
END FacialExpressionRecognition
```

Justification

Resizing the frame immediately after capture reduces the pixel count processed by every later stage, directly addressing limited computational resources. Using Haar cascades for detection and LBP histograms for feature extraction keeps both stages lightweight relative to deep-learning alternatives, sustaining real-time response. Modular functions (Capture, Detect, ExtractFeatures, Classify) allow any single stage to be optimized or swapped without restructuring the whole pipeline.